

Colab, or "Colaboratory", allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier. Watch [Introduction to Colab](#) or [Colab Features You May Have Missed](#) to learn more, or just get started below!

## ✓ Getting started

The document you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable, and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

```
86400
```

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut "Command/Ctrl+Enter". To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

```
604800
```

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To learn more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To learn more about the Jupyter project, see [jupyter.org](#).

## ✓ Data science

With Colab you can harness the full power of popular Python libraries to analyze and visualize data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualize it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from Github and many other sources. To learn more about importing data, and how Colab can be used for data science, see the links below under [Working with Data](#).

```
# =====
# Optical Image Formation, Spatial Sampling, and Aliasing Simulation Pipeline
# =====
# This notebook-ready script demonstrates how physical object visibility is
# governed by the optical Point Spread Function (PSF) and spatial sampling density.
#
# Pipeline Workflow:
# 1. Synthesizes a high-frequency, complex continuous test chart (Object).
# 2. Convolves the object with an optical lens PSF (Optical Degradation).
# 3. High-Resolution Sampling (Satisfies the Nyquist Criterion).
# 4. Low-Resolution Sampling (Violates Nyquist -> Causes Aliasing & Blur).
# =====

import numpy as np
import matplotlib.pyplot as plt
import cv2

# --- 1. GENERATE THE HIGH-FREQUENCY CONTINUOUS SCENE ---
# Define a high-resolution grid representing the "analog" continuous scene
dim = 1024
x = np.linspace(-10, 10, dim)
y = np.linspace(-10, 10, dim)
X, Y = np.meshgrid(x, y)

# Construct a zone plate (concentric circles with increasing spatial frequency)
```

```

r = np.sqrt(X**2 + Y**2)
analog_scene = np.sin(np.pi * r**2)
# Normalize to standard 8-bit image scale [0, 255]
analog_scene = ((analog_scene + 1) * 127.5).astype(np.float32)

# Overlay a tiny, high-contrast target object to demonstrate sub-pixel scale failure
# Drawing a thin, solid white cross at coordinates (250, 250)
cv2.line(analog_scene, (230, 250), (270, 250), 255, 3)
cv2.line(analog_scene, (250, 230), (250, 270), 255, 3)

# --- 2. OPTICAL IMAGE FORMATION: CONVOLUTION WITH LENS PSF ---
# Define a Gaussian Point Spread Function (PSF) representing optical lens blur
kernel_size = 31
sigma = 4.5
psf_kernel = cv2.getGaussianKernel(kernel_size, sigma)
psf_2d = np.outer(psf_kernel, psf_kernel)

# Convolve the true scene with the system's PSF: I_formed = I_true * PSF
optically_blurred_scene = cv2.filter2D(analog_scene, -1, psf_2d)

# --- 3. SPATIAL DISCRETIZATION & SAMPLING FREQUENCY ---

# Case A: High-Resolution Sampling (Dense Sensor Grid)
# We sample every 2nd pixel, keeping the sampling frequency high (fs > 2 * fmax)
high_res_step = 2
high_res_sampled = optically_blurred_scene[::high_res_step, ::high_res_step]

# Case B: Low-Resolution Sampling (Coarse Sensor Grid)
# We sample every 16th pixel, severely dropping below the Nyquist threshold
low_res_step = 16
low_res_sampled = optically_blurred_scene[::low_res_step, ::low_res_step]

# --- 4. DIGITAL SIGNAL RECONSTRUCTION FOR ANALYSIS ---
# To compare visibility side-by-side, we upscale the sampled matrices back to the
# original coordinate space using standard interpolation.

# High-Res structural reconstruction
high_res_reconstructed = cv2.resize(high_res_sampled, (dim, dim), interpolation=cv2.INTER_NEAREST)

# Low-Res structural reconstruction (Showcases aliasing patterns and pixelation)
low_res_reconstructed = cv2.resize(low_res_sampled, (dim, dim), interpolation=cv2.INTER_NEAREST)

# --- 5. DATA VISUALIZATION AND PLOTTING ---
fig, axes = plt.subplots(2, 2, figsize=(14, 13), dpi=100)
axes = axes.ravel()

# Plot 1: Perfect Continuous Scene
axes[0].imshow(analog_scene, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("1. Continuous Physical Scene\n(Contains Infinite Frequencies & Fine Cross)", fontsize=11, fontweight='bold')
axes[0].axis('off')

# Plot 2: Optical Blurring Image Formation
axes[1].imshow(optically_blurred_scene, cmap='gray', vmin=0, vmax=255)
axes[1].set_title("2. Optically Formed Image (Scene * PSF)\n(High-Frequency Wave Attenuation via Lens)", fontsize=11, fontweight='bold')
axes[1].axis('off')

# Plot 3: High-Resolution Uniform Sampling
axes[2].imshow(high_res_reconstructed, cmap='gray', vmin=0, vmax=255)
axes[2].set_title(f"3. High-Resolution Sampling Grid\n(Step={high_res_step}px | Object & Wave Structures Preserved)", fontsize=11, fontweight='bold')
axes[2].axis('off')

# Plot 4: Low-Resolution Sampling (Aliasing & Visibility Breakdown)
axes[3].imshow(low_res_reconstructed, cmap='gray', vmin=0, vmax=255)
axes[3].set_title(f"4. Low-Resolution Coarse Grid\n(Step={low_res_step}px | Destructive Spatial Aliasing & Blur)", fontsize=11, fontweight='bold')
axes[3].axis('off')

# Highlight the localized micro-object sub-pixel failure on the low-res plot
rect = plt.Rectangle((200, 200), 100, 100, edgecolor='red', facecolor='none', lw=2.5)
axes[3].add_patch(rect)
axes[3].text(150, 180, "Sub-Pixel Target Lost", color='red', fontsize=10, fontweight='bold')

plt.suptitle("Mathematical Mechanics of Spatial Sampling and Optical Image Formation", fontsize=15, fontweight='bold', y=0.9)
plt.subplots_adjust(wspace=0.1, hspace=0.2)
plt.show()

# --- 6. METRIC COMPUTATION AND LOGGING ---
# Calculate Structural Fidelity Loss via Mean Squared Error against the Formed Image
mse_high_res_vs_scene = np.mean((analog_scene - high_res_reconstructed)**2)

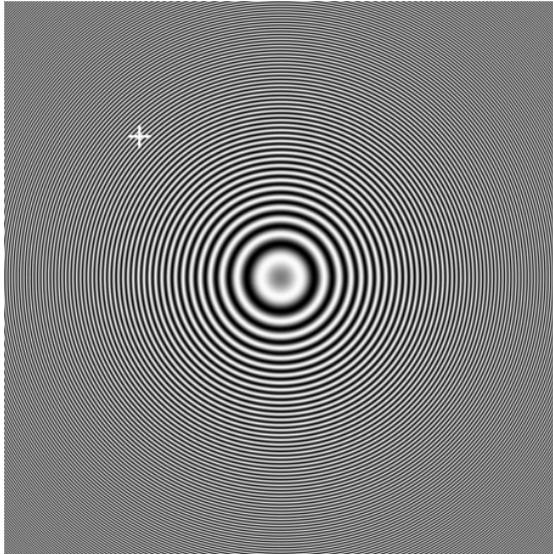
```

```
mse_high_res = np.mean((optically_blurred_scene - high_res_reconstructed) ** 2)
mse_low_res = np.mean((optically_blurred_scene - low_res_reconstructed) ** 2)

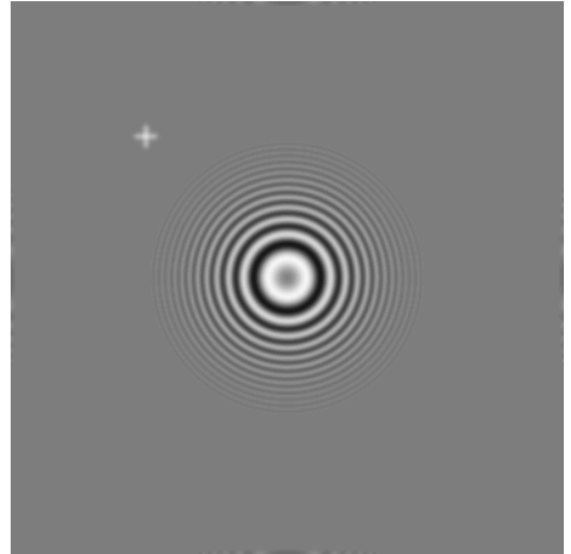
print("\n" + "="*70)
print("          SPATIAL DISCRETIZATION PERFORMANCE MATRIX          ")
print("="*70)
print(f"High-Res Grid Sampling Density (Matrix Dimensions) : {high_res_sampled.shape}")
print(f"High-Res Information Reconstruction Loss (MSE)      : {mse_high_res:.4f}")
print("-"*70)
print(f"Low-Res Grid Sampling Density (Matrix Dimensions)   : {low_res_sampled.shape}")
print(f"Low-Res Information Reconstruction Loss (MSE)       : {mse_low_res:.4f}")
print("="*70 + "\n")
print("CRITICAL OBSERVATION:")
print("Look closely at Subplot 4. Notice the coarse, unaligned geometric blocks and the ")
print("false low-frequency bands (Moiré patterns). The high-contrast micro-cross inside ")
print("the red box has been completely integrated into background pixels, demonstrating ")
print("total sub-pixel compression visibility failure.")
print("="*70 + "\n")
```

## Mathematical Mechanics of Spatial Sampling and Optical Image Formation

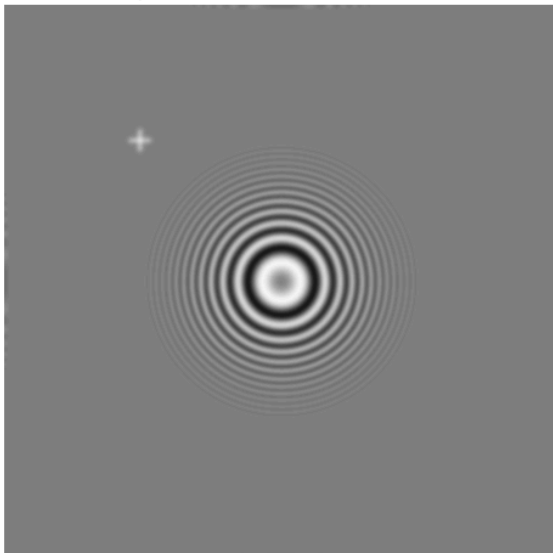
**1. Continuous Physical Scene**  
(Contains Infinite Frequencies & Fine Cross)



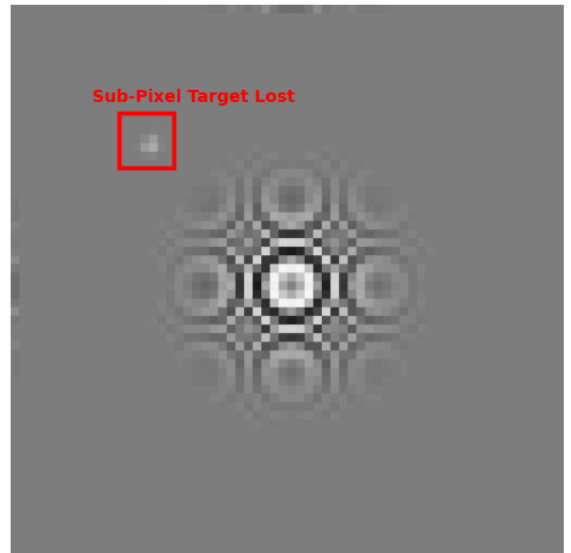
**2. Optically Formed Image (Scene \* PSF)**  
(High-Frequency Wave Attenuation via Lens)



**3. High-Resolution Sampling Grid**  
(Step=2px | Object & Wave Structures Preserved)



**4. Low-Resolution Coarse Grid**  
(Step=16px | Destructive Spatial Aliasing & Blur)



### SPATIAL DISCRETIZATION PERFORMANCE MATRIX

```
=====
High-Res Grid Sampling Density (Matrix Dimensions) : (512, 512)
High-Res Information Reconstruction Loss (MSE)      : 5.1298
=====
Low-Res Grid Sampling Density (Matrix Dimensions)  : (64, 64)
Low-Res Information Reconstruction Loss (MSE)      : 304.1858
=====
```

### CRITICAL OBSERVATION:

Look closely at Subplot 4. Notice the coarse, unaligned geometric blocks and the false low-frequency bands (Moiré patterns). The high-contrast micro-cross inside the red box has been completely integrated into background pixels, demonstrating total sub-pixel compression visibility failure.